

AWS GLUE DEV-TO-PROD CROSS-ACCOUNT DEPLOYMENT

Complete Start-Over Implementation Manual

Console-by-console instructions, exact field selections, IAM policies, scripts, validation and rollback

Document item	Value
Document version	1.0
Prepared date	20 July 2026
Target architecture	Dev CodeCommit -> Dev CodePipeline/CodeBuild -> Prod CodeCommit -> manual Prod CodeBuild -> AWS Glue
Current proof-of-concept Dev account	065320272077
Current proof-of-concept Prod account	811261691581
Current proof-of-concept Region	us-east-1
Use in other accounts	Replace every value enclosed in <ANGLE_BRACKETS> before implementation.

Read before implementation: Do not copy the proof-of-concept account IDs, role ARN or S3 bucket name into another environment. Complete the replacement worksheet in Phase 0 first.

Document contents

1. Purpose, scope and final architecture
2. Exact build order - what is first, second and next
3. Phase 0 - planning and dependency inventory
4. Phase 1 - Dev CodeCommit and Glue Studio source control
5. Phase 2 - Production repository and Production prerequisites
6. Phase 3 - Dev CodeBuild service role
7. Phase 4 - Production cross-account Git role
8. Phase 5 - Dev repository release files
9. Phase 6 - Dev CodePipeline and CodeBuild configuration
10. Phase 7 - first Dev-to-Prod repository promotion
11. Phase 8 - Production deployment service role
12. Phase 9 - Production deployment framework files
13. Phase 10 - Production CodeBuild project
14. Phase 11 - first Production Glue deployment
15. Phase 12 - normal release, hotfix and rollback procedures
16. Troubleshooting by error message
17. Security, dependencies, limitations and scaling
18. Appendix - complete scripts, IAM policies and AWS references

How to use this manual: Complete one phase at a time. Do not move to the next phase until every item in the phase-completion checklist is checked and evidence is recorded.

1. Purpose, scope and final architecture

1.1 Purpose

This manual recreates the complete selective AWS Glue job promotion solution in a new pair of AWS accounts. It is written so an engineer can start with empty CI/CD resources and follow the steps in order without referring to the original chat or proof of concept.

1.2 Final architecture

```

DEV ACCOUNT

AWS Glue Studio
| Push to repository
v
Dev CodeCommit repository
| contains JobName/JobName.py + JobName/JobName.json
| release-jobs.json chooses approved folders
v
Dev CodePipeline - manually started
| Source action supplies repository files
v
Dev CodeBuild - GlueReleaseBuilder
| assumes GlueReleasePushRole in Production
| copies only approved folders
v

PRODUCTION ACCOUNT

Prod CodeCommit - glue-production
| Production-approved source history
| deployment-jobs.txt chooses jobs for this deployment
v
Prod CodeBuild - GlueProductionDeployer - manually started
| uploads selected scripts to Prod S3
| CreateJob if absent / UpdateJob if present
v
Production AWS Glue jobs
  
```

1.3 What the solution changes during deployment

Glue property	Production action
IAM execution role	Replaced with the Production Glue execution role.
Command.ScriptLocation	Replaced with s3://<PROD_GLUE_ASSETS_BUCKET>/scripts/<JobName>.py.
--spark-event-logs-path	Replaced with the Production sparkHistoryLogs prefix.
--TempDir	Replaced with the Production temporary prefix.
Other common Glue job settings	Copied from the Glue Studio JSON when supported by CreateJob/UpdateJob.
SourceControlDetails	Intentionally not copied, so Production does not point back to the Dev repository.

1.4 Scope

- Promotion of selected Glue job source folders from Dev CodeCommit to Prod CodeCommit.
- Creation or update of selected Glue jobs in Production.
- Cross-account IAM required for the Dev build to push to the Production repository.
- Production script upload into the Production Glue assets bucket.
- Manual release and deployment operation with phase checklists and rollback.

1.5 Out of scope - create these separately

- Glue connections and their network configuration.

- VPCs, subnets, route tables, security groups, NAT gateways and VPC endpoints.
- Glue Data Catalog databases and tables, crawlers, workflows, triggers and schedules.
- Secrets Manager secrets, SSM parameters, KMS keys and certificates.
- Job-specific source and target S3 buckets and their bucket policies.
- Lake Formation grants and database permissions.
- Downstream orchestration such as Step Functions unless added as a separate deployment scope.

Important dependency rule: A Glue job can be created successfully but fail at runtime when a referenced connection, security configuration, secret, database, table, library, bucket or network route does not exist in Production.

2. Exact build order - what is first, second and next

Order	Account	Action	Do not continue until
1	Both	Record account IDs, Region, role, bucket and repository names.	The Phase 0 worksheet is complete.
2	Dev	Create the Dev CodeCommit repository and main branch.	README or first commit exists.
3	Dev	Connect a sample Glue job to Dev CodeCommit and push .py + .json.	The exact job folder is visible.
4	Prod	Create the Prod CodeCommit repository and main branch.	README exists in main.
5	Prod	Confirm/create the Production Glue assets bucket and prefixes.	scripts, temporary and sparkHistoryLogs are ready.
6	Prod	Confirm the Production Glue execution role.	The exact role ARN is recorded.
7	Dev	Create the Dev CodeBuild service role.	The exact role ARN is recorded.
8	Prod	Create GlueReleasePushRole trusting the Dev CodeBuild role.	Trust and CodeCommit permissions validate.
9	Dev	Add release-jobs.json, release.py and buildspec.yml.	Files exist on the Dev main branch.
10	Dev	Create CodePipeline and create CodeBuild from the Build stage.	CodeBuild source displays CodePipeline.
11	Dev	Disable the EventBridge source rule if releases must be manual.	A commit alone does not start the pipeline.
12	Dev/Prod	Run the first selective promotion.	Only approved job folders exist in Prod CodeCommit.
13	Prod	Create Production CodeBuild service role.	Role has CodeCommit, logs, S3, Glue and PassRole permissions.
14	Prod	Add deployment-jobs.txt, glue_config.py, deploy.py and buildspec.yml.	All four files exist in Prod main.
15	Prod	Create standalone Production CodeBuild with CodeCommit source.	Project source is Prod CodeCommit main.
16	Prod	Run the first deployment manually.	Selected Glue jobs are created or updated.
17	Prod	Validate four Production overrides and runtime dependencies.	Evidence is captured and the change is closed.

Why the Dev role is created before the Prod cross-account role: The Production role trust policy must contain the exact ARN of the Dev CodeBuild role. Creating the Dev role first avoids broad temporary trust and avoids editing the trust twice.

3. Phase 0 - planning and dependency inventory

Step 0.1 - Complete the replacement worksheet

Item	Details	
Account	Both	
AWS service	IAM, Glue, S3, CodeCommit	
Objective	Create the values that will be used in every policy and script.	
Console navigation	AWS Console account menu, Region selector and service consoles	
Placeholder	Enter the new value	Proof-of-concept example
<REGION>		us-east-1
<DEV_ACCOUNT_ID>		065320272077
<PROD_ACCOUNT_ID>		811261691581
<DEV_REPOSITORY_NAME>		glue-development
<PROD_REPOSITORY_NAME>		glue-production
<DEV_BRANCH>		main
<PROD_BRANCH>		main
<DEV_CODEBUILD_ROLE_NAME>		GlueReleaseBuilderRole
<PROD_RELEASE_ROLE_NAME>		GlueReleasePushRole
<PROD_CODEBUILD_ROLE_NAME>		CodeBuildGlueProductionDeployerRole
<PROD_GLUE_ROLE_ARN>		arn:aws:iam::811261691581:role/Glue-role-2d766430
<PROD_GLUE_ASSETS_BUCKET>		aws-glue-assets-811261691581-us-east-1

Step 0.2 - Inventory every selected Glue job dependency

Item	Details
Account	Dev and Prod
AWS service	AWS Glue and dependent services
Objective	Prevent jobs from being deployed before required Production resources exist.
Console navigation	Glue > ETL jobs > open job > Job details and Script

19. Choose the first two jobs for the implementation test. Do not begin with all 200 jobs.
20. Open each Dev job and record every resource referenced in Job details, connections and default arguments.
21. Open the script and search for s3://, secret names, parameter names, database names, table names, connection names and hard-coded account IDs.
22. Confirm the equivalent resource exists in Production or record an owner and completion date.
23. Repeat for every job added to a release batch.

Dependency	Dev value	Prod value	Ready?	Owner / action
Glue connection			[]	

Dependency	Dev value	Prod value	Ready?	Owner / action
Glue security configuration			☐	
Source S3 bucket/prefix			☐	
Target S3 bucket/prefix			☐	
Secrets Manager / SSM parameter			☐	
KMS key			☐	
Catalog database/table			☐	
Extra Python files / JARs			☐	
VPC/subnet/security group route			☐	
Lake Formation grants			☐	

Step 0.3 - Confirm administrator and service availability

Item		Details
Account		Both
AWS service		IAM and AWS Developer Tools
Objective		Ensure the implementation can be completed in both accounts.
Console navigation		IAM > Roles; CodeCommit; CodeBuild; CodePipeline; Glue; S3
Check	Completion criterion	Evidence / value
☐	You can create and edit IAM roles and inline policies in Dev.	
☐	You can create and edit IAM roles and inline policies in Production.	
☐	CodeCommit, CodeBuild and CodePipeline are available in the selected Region.	
☐	You can create files and commits in both CodeCommit repositories.	
☐	You can open Glue Studio and push a job to source control in Dev.	
☐	You can start CodeBuild and CodePipeline executions and read CloudWatch logs.	
☐	Organization SCPs and permission boundaries do not deny sts:AssumeRole, CodeCommit Git, Glue CreateJob/UpdateJob, S3 PutObject or iam:PassRole.	
Current CodeCommit status: AWS CodeCommit returned to general availability and new customer sign-ups reopened in November 2025. Confirm the selected Region supports CodeCommit before implementation. Reference AWS-12.		

Phase 0 completion checklist

Check	Completion criterion	Evidence / value
☐	All placeholders have final values.	
☐	Two test Glue jobs are selected.	
☐	Dependency inventory is complete for both test jobs.	
☐	Production Glue role and assets bucket approach are decided.	
☐	Permissions/SCP blockers have been resolved.	

4. Phase 1 - Dev CodeCommit and Glue Studio source control

Step 1.1 - Create the Dev CodeCommit repository

Item		Details
Account		Dev
AWS service		AWS CodeCommit
Objective		Create the source-of-truth repository for Dev Glue Studio job files.
Console navigation		AWS Console > CodeCommit > Repositories > Create repository
Prerequisites		Phase 0 worksheet is complete.
Console field	Select / enter	Why / leave as
Repository name	<DEV_REPOSITORY_NAME>	Use a stable name; repository names are case-sensitive.
Description	Development AWS Glue job source and release controls	Recommended.
Tags	Environment=Dev; Application=Glue-CICD	Use your organization tagging standard.
Encryption	AWS managed key	Use a customer managed key only when required by policy.
CodeGuru Reviewer	Disabled	Not required for this deployment process.

24. Confirm the console is in <REGION>.

25. Choose Create repository.

26. Enter the fields above.

27. Choose Create.

28. On the empty repository page choose Add file > Create file.

29. File name: README.md.

30. Enter a short description such as "Development Glue source repository".

31. Author name: your team or change owner.

32. Email address: a valid team email.

33. Commit message: Initialize main branch.

34. Choose Commit changes.

Expected result: The repository opens on branch main and contains README.md. Reference AWS-1.

Do not leave the repository completely empty: Creating the first file establishes the main branch and removes branch ambiguity for Glue Studio, CodePipeline and Git clone operations.

Step 1.2 - Confirm Glue Studio source-control permissions

Item		Details
Account		Dev
AWS service		IAM and AWS Glue
Objective		Allow the engineer to push/pull Glue jobs to CodeCommit.
Console navigation		IAM > the console user or role used for Glue Studio
Prerequisites		The Dev repository and main branch exist.

- At minimum, Glue Git integration uses glue:UpdateSourceControlFromJob, glue:UpdateJobFromSourceControl, s3:GetObject and s3:PutObject.
- The console identity also needs CodeCommit read/write permissions to the Dev repository.
- Administrator access already includes these permissions, but production implementations should use least privilege.

AWS reference: AWS Glue documents the Version Control tab, required folder/file naming and minimum Glue/S3 permissions. Reference AWS-2.

Step 1.3 - Connect the first Dev Glue job to CodeCommit

Item	Details	
Account	Dev	
AWS service	AWS Glue Studio	
Objective	Create the required JobName/JobName.py and JobName/JobName.json files.	
Console navigation	Glue > ETL jobs > select the job > Version control	
Prerequisites	A tested Dev Glue job exists and the Dev repository has main.	
Console field	Select / enter	Why / leave as
Version control system	AWS CodeCommit	Do not choose GitHub/GitLab/Bitbucket for this design.
Repository	<DEV_REPOSITORY_NAME>	Must be the Dev repository in the same account/Region.
Branch	main	Use the branch selected in Phase 0.
Folder	Exact Glue job name, for example Demo-copy	The folder and JSON name must match the job name exactly.

35. Open the Glue job and choose the Version control tab.
36. For Version control system choose AWS CodeCommit.
37. Choose the Dev repository and branch main.
38. In Folder enter the exact case-sensitive Glue job name.
39. Choose Save on the Glue job.
40. Open Actions and choose Push to repository.
41. Choose Confirm.
42. Open the link to the CodeCommit commit if Glue displays it.
43. In CodeCommit confirm the job folder contains JobName.py and JobName.json.

Expected result: The Dev repository contains <JobName>/<JobName>.py and <JobName>/<JobName>.json. Glue Studio creates a repository commit when Push to repository is confirmed.

Push overwrites the repository version: AWS Glue Studio pushes the last saved job and can overwrite a repository copy that was changed elsewhere. Review differences before pushing. Reference AWS-2.

Step 1.4 - Repeat source-control setup for the second test job

Item	Details
Account	Dev
AWS service	AWS Glue Studio
Objective	Prove the folder naming pattern works for multiple jobs.
Console navigation	Repeat Step 1.3

Item	Details
Prerequisites	The first test job is visible in CodeCommit.

44. Open the second tested Dev Glue job.
45. Use the same repository and branch.
46. Use the exact second job name as the folder.
47. Save and Push to repository.
48. Confirm both folders exist independently in CodeCommit.

Phase 1 completion checklist

Check	Completion criterion	Evidence / value
☐	Dev CodeCommit repository exists in the correct Region.	
☐	main branch exists.	
☐	README.md exists.	
☐	First test job folder contains matching .py and .json.	
☐	Second test job folder contains matching .py and .json.	
☐	No job folder is named differently from the Glue job.	

5. Phase 2 - Production repository and Production prerequisites

Step 2.1 - Create the Production CodeCommit repository

Item		Details
Account		Prod
AWS service		AWS CodeCommit
Objective		Create the controlled Production-approved code repository.
Console navigation		CodeCommit > Repositories > Create repository
Prerequisites		The Production account ID and Region are confirmed.
Console field	Select / enter	Why / leave as
Repository name	<PROD_REPOSITORY_NAME> (recommended: glue-production)	This exact value is used by release.py and IAM.
Description	Production-approved AWS Glue job source	Recommended.
Tags	Environment=Prod; Application=Glue-CICD	Use the organization standard.
Encryption	AWS managed key	Use a customer managed key only when mandated.
CodeGuru Reviewer	Disabled	Not required.

49. Create the repository using the field values above.

50. Choose Add file > Create file.

51. Create README.md in branch main.

52. Author and commit message: Initialize Production repository.

53. Choose Commit changes.

54. Record the repository ARN from Settings or construct
arn:aws:codecommit:<REGION>:<PROD_ACCOUNT_ID>:<PROD_REPOSITORY_NAME>.

Expected result: The Prod repository contains only README.md on main. No Glue job folders have been copied manually.

Step 2.2 - Confirm or create the Production Glue assets bucket

Item		Details
Account		Prod
AWS service		Amazon S3
Objective		Provide the Production script, temporary and Spark event-log locations.
Console navigation		S3 > Buckets
Prerequisites		The selected bucket name is globally unique and recorded.
Preferred option: Use the existing aws-glue-assets-<PROD_ACCOUNT_ID>-<REGION> bucket when it already exists and meets security requirements. Otherwise create a dedicated bucket such as prod-glue-assets-<PROD_ACCOUNT_ID>-<REGION>.		
Console field	Select / enter	Why / leave as
Bucket name	<PROD_GLUE_ASSETS_BUCKET>	Must be globally unique.

Item		Details
AWS Region	<REGION>	Use the same Region as Glue and CodeBuild.
Object Ownership	ACLs disabled / Bucket owner enforced	Recommended.
Block Public Access	Block all public access	Must remain enabled.
Bucket Versioning	Enable	Strongly recommended for script rollback.
Default encryption	SSE-S3 or approved KMS key	Follow organization policy.
Object Lock	Disabled unless formally required	Cannot be casually removed after enablement.
Tags	Environment=Prod; Purpose=GlueAssets	Recommended.

55. Create or open the selected bucket.

56. Create the folder/prefix scripts/.

57. Create temporary/.

58. Create sparkHistoryLogs/.

59. Record the bucket name exactly without s3:// and without a trailing slash.

60. If a KMS key is used, record its ARN and add required KMS permissions to the Glue role and Prod CodeBuild role.

Expected result: The Production bucket is private, encrypted and has scripts, temporary and sparkHistoryLogs prefixes.

Step 2.3 - Confirm the Production Glue execution role

Item	Details
Account	Prod
AWS service	IAM and AWS Glue
Objective	Identify the role every deployed Glue job will use.
Console navigation	IAM > Roles > open the role; Glue > existing job > Job details
Prerequisites	Production data permissions are understood.

61. Open the intended Production Glue role.

62. On Trust relationships confirm glue.amazonaws.com can assume it.

63. Confirm it can read the deployed scripts in the Production assets bucket.

64. Confirm job-specific source/target, secrets, connections, KMS and logging permissions.

65. Copy the full role ARN into the Phase 0 worksheet.

66. Do not use the Dev Glue role ARN.

Baseline Glue role trust policy:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "glue.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

Role permissions are job-specific: This manual cannot safely invent data-access permissions for all 200 jobs. The Production Glue role must be designed around the actual buckets, connections, secrets, databases and KMS keys used by those jobs.

Phase 2 completion checklist

Check	Completion criterion	Evidence / value
☐	Prod CodeCommit repository exists with main and README.md.	
☐	Production repository ARN is recorded.	
☐	Production assets bucket is private, encrypted and versioned.	
☐	scripts, temporary and sparkHistoryLogs prefixes exist.	
☐	Production Glue role ARN is recorded.	
☐	Production Glue role trust includes glue.amazonaws.com.	
☐	Dependency owners have confirmed the role has required data permissions.	

6. Phase 3 - Dev CodeBuild service role

Step 3.1 - Create GlueReleaseBuilderRole

Item		Details
Account		Dev
AWS service		AWS IAM
Objective		Create the exact Dev role that Production will trust.
Console navigation		IAM > Roles > Create role
Prerequisites		Dev account ID, Prod account ID and role names are final.
Console field	Select / enter	Why / leave as
Trusted entity type	AWS service	This is a CodeBuild service role.
Use case	CodeBuild	Creates the codebuild.amazonaws.com trust.
Permissions policies	Do not attach broad managed policies	Add the exact inline policy after role creation.
Role name	GlueReleaseBuilderRole	Use the same name in the Prod trust policy.
Description	Dev CodeBuild role for selective Glue promotion	Recommended.
Tags	Environment=Dev; Application=Glue-CICD	Recommended.

67. Choose AWS service and CodeBuild.

68. Choose Next.

69. Do not attach AdministratorAccess or AWSCodeCommitPowerUser.

70. Choose Next.

71. Enter GlueReleaseBuilderRole.

72. Choose Create role.

73. Open the new role.

74. Copy the exact Role ARN. If the role was created manually, the path normally appears as
arn:aws:iam::<DEV_ACCOUNT_ID>:role/GlueReleaseBuilderRole.

Expected result: The role exists and trusts codebuild.amazonaws.com, but it has no unnecessary broad permissions.

Step 3.2 - Attach the Dev CodeBuild inline permissions

Item		Details
Account		Dev
AWS service		AWS IAM
Objective		Allow build logs and assumption of the Production release role.
Console navigation		IAM > Roles > GlueReleaseBuilderRole > Permissions > Add permissions > Create inline policy
Prerequisites		The Production role name is already selected.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "WriteCodeBuildLogs",
```

```

"Effect": "Allow",
"Action": [
  "logs:CreateLogGroup",
  "logs:CreateLogStream",
  "logs:PutLogEvents"
],
"Resource": "arn:aws:logs:<REGION>:<DEV_ACCOUNT_ID>:log-group:/aws/codebuild/GlueReleaseBuilder*"
},
{
  "Sid": "AssumeProductionReleaseRole",
  "Effect": "Allow",
  "Action": "sts:AssumeRole",
  "Resource": "arn:aws:iam::<PROD_ACCOUNT_ID>:role/GlueReleasePushRole"
}
]
}

```

75. Replace <REGION>, <DEV_ACCOUNT_ID> and <PROD_ACCOUNT_ID>.
76. Choose the JSON policy editor and paste the policy.
77. Choose Next.
78. Policy name: GlueReleaseBuilderPermissions.
79. Choose Create policy.
80. Re-open the role permissions and confirm the inline policy is attached.

The Prod role may not exist yet: The IAM policy can reference a role ARN before the role is created. The build cannot assume it until Phase 4 is complete.

Phase 3 completion checklist

Check	Completion criterion	Evidence / value
[]	GlueReleaseBuilderRole exists in Dev.	
[]	Trust principal is codebuild.amazonaws.com.	
[]	Role ARN is copied exactly.	
[]	Inline policy allows only CodeBuild logs and sts:AssumeRole to the selected Prod role.	
[]	No administrator or broad CodeCommit managed policy is attached.	

7. Phase 4 - Production cross-account Git role

Step 4.1 - Create GlueReleasePushRole trust policy

Item	Details
Account	Prod
AWS service	AWS IAM
Objective	Trust only the exact Dev CodeBuild role.
Console navigation	IAM > Roles > Create role > Custom trust policy
Prerequisites	The exact Dev role ARN from Phase 3 is available.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "TrustDevCodeBuildRole",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<DEV_ACCOUNT_ID>:role/service-role/GlueReleaseBuilderRole"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

81. Replace <DEV_ACCOUNT_ID>.
82. If the Dev role ARN contains /service-role/, include that exact path; if it does not, do not add it.
83. In Create role choose Custom trust policy.
84. Paste the policy.
85. Choose Next.
86. Do not attach a broad policy on the permissions page.
87. Role name: GlueReleasePushRole.
88. Description: Allows Dev Glue release CodeBuild to update the Production Glue repository.
89. Choose Create role.

Expected result: The trust policy Principal is the exact Dev CodeBuild role ARN, not the Dev account root and not *.

Step 4.2 - Attach repository-scoped CodeCommit permissions

Item	Details
Account	Prod
AWS service	AWS IAM
Objective	Permit Git pull/push only to the Production Glue repository.
Console navigation	IAM > Roles > GlueReleasePushRole > Create inline policy
Prerequisites	The Prod repository ARN is known.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadAndPushProductionRepository",
      "Effect": "Allow",
      "Action": [
        "codecommit:GetRepository",
        "codecommit:GetBranch",
        "codecommit:GitPull",
        "codecommit:GitPush"
      ],
      "Resource": "arn:aws:codecommit:<REGION>:<PROD_ACCOUNT_ID>:glue-production"
    }
  ]
}
```

```
}
}
}
```

90. Replace <REGION> and <PROD_ACCOUNT_ID>.
91. Replace glue-production if another Prod repository name is used.
92. Paste the JSON as an inline policy.
93. Policy name: GlueProductionRepositoryGitAccess.
94. Choose Create policy.

Why both policies are required: Cross-account access has two halves: the Prod role trust permits the Dev role to assume it, and the Prod role permission policy defines what temporary credentials can do. Reference AWS-6.

Step 4.3 - Verify the two-sided cross-account configuration

Item		Details
Account		Dev and Prod
AWS service		AWS IAM
Objective		Confirm both sides contain matching exact ARNs.
Console navigation		Compare Dev role permission and Prod role trust
Prerequisites		Both inline policies are saved.
Check	Completion criterion	Evidence / value
[]	Dev role policy Resource equals arn:aws:iam::<PROD_ACCOUNT_ID>:role/GlueReleasePushRole.	
[]	Prod trust Principal equals the exact Dev GlueReleaseBuilderRole ARN.	
[]	Prod repository policy Resource equals the exact CodeCommit repository ARN.	
[]	No wildcard Principal is used.	
[]	No unrelated Production repositories are granted.	

Phase 4 completion checklist

Check	Completion criterion	Evidence / value
[]	GlueReleasePushRole exists in Production.	
[]	Trust policy contains the exact Dev role ARN.	
[]	Inline policy contains GitPull, GitPush, GetRepository and GetBranch for only the Prod repository.	
[]	Dev role and Prod role ARNs match on both sides.	

8. Phase 5 - Dev repository release files

Step 5.1 - Create release-jobs.json

Item	Details
Account	Dev
AWS service	AWS CodeCommit
Objective	Choose the exact Glue folders allowed to enter the Production repository.
Console navigation	CodeCommit > <DEV_REPOSITORY_NAME> > Code > Add file > Create file
Prerequisites	Two job folders already exist.

```
{
  "release": "1.0.0",
  "description": "Initial Production release",
  "jobs": [
    "Demo-copy"
  ]
}
```

95. Create release-jobs.json in the repository root.
96. Replace Demo-copy with the first approved test job folder.
97. For the initial controlled test, list one job only.
98. Author name and email: release owner.
99. Commit message: Add Glue release manifest.
100. Choose Commit changes.

This is the promotion control: A job that is present in Dev CodeCommit but absent from release-jobs.json must not be copied to Production.

Step 5.2 - Create release.py

Item	Details
Account	Dev
AWS service	AWS CodeCommit
Objective	Run the selective cross-account Git promotion.
Console navigation	CodeCommit > Dev repo > Add file > Create file
Prerequisites	release-jobs.json exists.

101. Create release.py at repository root.
102. Copy the complete script from Appendix A or the project-template ZIP.
103. Do not hard-code access keys.
104. Do not hard-code the Prod account ID in the script when using environment variables.
105. Commit message: Add selective Glue release script.

Environment variables used by release.py: PROD_RELEASE_ROLE_ARN, PROD_REPOSITORY_NAME and PROD_BRANCH. AWS_REGION is supplied automatically by CodeBuild.

Step 5.3 - Create the Dev buildspec.yml

Item	Details
Account	Dev

Item	Details
AWS service	AWS CodeCommit
Objective	Tell CodeBuild how to validate and run release.py.
Console navigation	CodeCommit > Dev repo > Add file > Create file
Prerequisites	release.py exists.

```
version: 0.2
```

```
phases:
```

```
install:
```

```
runtime-versions:
```

```
python: 3.11
```

```
pre_build:
```

```
commands:
```

```
- echo "Validating release files"
```

```
- python3 -m py_compile release.py
```

```
- test -f release-jobs.json
```

```
build:
```

```
commands:
```

```
- echo "Starting selective Glue release to the Production repository"
```

```
- python3 release.py
```

106. Create buildspec.yml in the repository root.

107. Paste the YAML exactly with spaces, not tabs.

108. Commit message: Add Dev Glue release buildspec.

Buildspec location: When CodeBuild is configured to Use a buildspec file, it looks for buildspec.yml in the source root by default. Reference AWS-4.

Step 5.4 - Review the final Dev repository structure

Item	Details
Account	Dev
AWS service	AWS CodeCommit
Objective	Confirm the Source stage will contain all release files.
Console navigation	CodeCommit > Dev repo > Code > main
Prerequisites	All files are committed.

```
<DEV_REPOSITORY_NAME>/
```

```
|-- README.md
```

```
|-- Demo-copy/
```

```
| |-- Demo-copy.py
```

```
| |-- Demo-copy.json
```

```
|-- SecondJob/
```

```
| |-- SecondJob.py
```

```
| |-- SecondJob.json
```

```
|-- release-jobs.json
```

```
|-- release.py
```

```
`-- buildspec.yml
```

Phase 5 completion checklist

Check	Completion criterion	Evidence / value
[]	release-jobs.json is valid JSON.	
[]	The jobs array contains one approved test job.	
[]	release.py is in the repository root.	

Check	Completion criterion	Evidence / value
☐	buildspec.yml is in the repository root.	
☐	Every listed folder contains matching .py and .json files.	
☐	No credentials are stored in source control.	

9. Phase 6 - Dev CodePipeline and CodeBuild configuration

Source selection rule: For the Dev release project used by CodePipeline, create or select the CodeBuild project from the CodePipeline Build stage. The CodeBuild source should display CodePipeline. When a project is created directly in the CodeBuild console, CodePipeline is not offered as a source provider; that behavior is expected. Reference AWS-5.

Step 6.1 - Start the Dev CodePipeline wizard

Item		Details
Account		Dev
AWS service		AWS CodePipeline
Objective		Create a manually operated release pipeline.
Console navigation		CodePipeline > Pipelines > Create pipeline
Prerequisites		Dev repository files and both IAM roles are complete.
Console field	Select / enter	Why / leave as
Creation option	Build custom pipeline	Do not choose a template that adds unrelated deployment actions.
Pipeline name	GlueReleasePipeline1	Use the approved name.
Pipeline type	V2	Matches the proof of concept and supports Queued mode.
Execution mode	Queued	Prevents two releases from changing Prod repository concurrently.
Service role	New service role	Allow CodePipeline to create a scoped role.
Role name	Accept generated name or use AWSCodePipelineServiceRole-<REGION>- GlueReleasePipeline1	Record the role.
Artifact store	Default location	CodePipeline creates/uses an artifact S3 bucket in Dev.
Encryption key	AWS managed key	Use customer KMS only when required.
Variables	None	Not needed for this implementation.
Triggers	Default during creation; disable EventBridge after validation	Manual-only operation is configured after creation.

109. Open CodePipeline in <REGION>.

110. Choose Create pipeline.

111. Choose Build custom pipeline.

112. Enter the values above.

113. Choose Next.

Step 6.2 - Configure the Source stage

Item	Details
Account	Dev
AWS service	AWS CodePipeline
Objective	Supply the Dev repository files to the build.

Item		Details
Console navigation		Pipeline wizard > Add source stage
Prerequisites		Pipeline properties are complete.
Console field	Select / enter	Why / leave as
Source provider	AWS CodeCommit	Dev source repository.
Repository name	<DEV_REPOSITORY_NAME>	Select the Dev repository.
Branch name	main	Select the Dev release branch.
Output artifact format	CodePipeline default / CODE_ZIP	The build needs files, not Git history.
Output artifact name	SourceArtifact	Use the generated/default value.
Change detection	Console default initially	The console creates an EventBridge rule; disable it later for manual-only operation.

114. Choose AWS CodeCommit.

115. Select repository and branch.

116. Leave output format as CodePipeline default.

117. Choose Next.

Step 6.3 - Create the Dev CodeBuild project from the Build stage

Item	Details
Account	Dev
AWS service	AWS CodePipeline and CodeBuild
Objective	Create GlueReleaseBuilder with CodePipeline as source.
Console navigation	Pipeline wizard > Add build stage > Build provider AWS CodeBuild > Create project
Prerequisites	The Dev CodeBuild IAM role already exists.

118. For Build provider choose AWS CodeBuild.

119. Region: <REGION>.

120. Choose Create project. A CodeBuild configuration window opens.

121. Complete every field in the following tables.

CodeBuild - Project configuration

Console field	Select / enter	Why / leave as
Project name	GlueReleaseBuilder	Exact project name used in logs and documentation.
Description	Promotes selected Dev Glue folders to Prod CodeCommit	Recommended.
Build badge	Disabled	Not required.
Concurrent build limit	1	Prevents simultaneous Git pushes to the same Prod branch.
Tags	Environment=Dev; Application=Glue-CICD	Recommended.

CodeBuild - Source

Console field	Select / enter	Why / leave as
Source provider	CodePipeline	This is set automatically because the project is created from the pipeline.
Primary source webhook	Not applicable	CodePipeline controls execution.
Git clone depth	Not applicable to ZIP input	Source action supplies an artifact.
Build status reporting	Disabled / not applicable	Not needed.

CodeBuild - Environment

Console field	Select / enter	Why / leave as
Provisioning model	On-demand	No continuously billed reserved fleet.
Environment image	Managed image	Use AWS-maintained image.
Compute	EC2	Compatible with Git and AWS CLI workflow.
Operating system	Amazon Linux	Use current Amazon Linux managed image.
Runtime(s)	Standard	Provides Git, AWS CLI and Python.
Image	aws/codebuild/amazonlinux-x86_64-standard:5.0 or current approved equivalent	Use the latest version in the selected runtime family.
Image version	Always use the latest image for this runtime version	Receives patched image revisions.
Environment type	Linux EC2	Default x86_64.
Compute type	Small / BUILD_GENERAL1_SMALL	Sufficient for JSON validation and Git copy.
Service role	Existing service role	The role was created in Phase 3.
Role ARN	arn:aws:iam::<DEV_ACCOUNT_ID>:role/GlueReleaseBuilderRole	Select the exact role.
Privileged mode	Disabled	No Docker build is performed.
Certificate	None	Not required.
VPC	No VPC	The build uses public AWS service endpoints. Configure VPC only when organization policy requires endpoints/NAT.
Auto-retry limit	0	A failed release should be reviewed, not automatically repeated.
Timeout	60 minutes	Default and more than enough for selected folders.
Queued timeout	Default	No change required.

CodeBuild - Environment variables

Name	Value	Type
PROD_RELEASE_ROLE_ARN	arn:aws:iam::<PROD_ACCOUNT_ID>:role/GlueReleasePushRole	Plaintext

Name	Value	Type
PROD_REPOSITORY_NAME	<PROD_REPOSITORY_NAME>	Plaintext
PROD_BRANCH	main	Plaintext

Security: These variables are identifiers, not secrets. Never place AWS access keys in CodeBuild environment variables or repository files.

CodeBuild - Buildspec, batch, artifacts, cache and logs

Console field	Select / enter	Why / leave as
Build specifications	Use a buildspec file	The file is committed in the repository.
Buildspec name	buildspec.yml or leave blank	Blank uses root buildspec.yml.
Batch configuration	Disabled	One release build is required.
Artifacts	CodePipeline / no explicit output artifact	The build performs a Git push and does not produce a deployment package.
Cache type	No cache	The build is small and deterministic.
CloudWatch logs	Enabled	Required for evidence and troubleshooting.
CloudWatch group name	/aws/codebuild/GlueReleaseBuilder or leave generated	Use an explicit group if your standard requires.
CloudWatch stream name	Leave generated	Build ID identifies the stream.
S3 logs	Disabled	CloudWatch Logs is sufficient unless policy requires archive.

122. Choose Continue to CodePipeline.

123. Back in the Build stage select the newly created GlueReleaseBuilder project.

124. Input artifact should be SourceArtifact.

125. Output artifact is not required.

126. Choose Next.

Step 6.4 - Skip the Deploy stage and create the pipeline

Item	Details
Account	Dev
AWS service	AWS CodePipeline
Objective	Finish the Dev promotion pipeline without deploying Glue directly.
Console navigation	Pipeline wizard > Add deploy stage
Prerequisites	Source and Build stages are configured.

127. Choose Skip deploy stage.

128. Confirm Skip when prompted.

129. Review pipeline name, Source action and Build action.

130. Choose Create pipeline.

131. The first execution starts automatically after creation.

132. Open the Build action and follow View details to CodeBuild logs.

Expected result: The first execution reaches the build. It should assume the Prod role, clone the Prod repository, copy the one approved folder, commit and push.

Step 6.5 - Disable automatic EventBridge triggering for manual-only release

Item	Details
Account	Dev
AWS service	Amazon EventBridge and CodePipeline
Objective	Ensure commits do not automatically promote code.
Console navigation	EventBridge > Rules; CodePipeline > GlueReleasePipeline1
Prerequisites	The first pipeline execution has completed or been reviewed.

133. Open EventBridge in <REGION>.

134. Choose Rules on the default event bus.

135. Locate the rule whose target is GlueReleasePipeline1 and whose event pattern references the Dev CodeCommit repository.

136. Open the rule and choose Disable.

137. Return to CodePipeline.

138. Make a harmless documentation commit in Dev and confirm the pipeline does not start automatically.

139. To release, open GlueReleasePipeline1 and choose Release change / Start pipeline execution manually.

Do not delete the pipeline artifact bucket: The artifact bucket is required even though the CodeBuild action does not output a deployment package. CodePipeline stores its source artifact there. Reference AWS-7.

Phase 6 completion checklist

Check	Completion criterion	Evidence / value
☐	Pipeline GlueReleasePipeline1 exists.	
☐	Pipeline type V2 and execution mode Queued are set.	
☐	Source provider is CodeCommit Dev repository main.	
☐	CodeBuild project source displays CodePipeline.	
☐	Existing GlueReleaseBuilderRole is selected.	
☐	All three environment variables are correct.	
☐	Privileged mode and VPC are disabled.	
☐	Buildspec is buildspec.yml.	
☐	CloudWatch logs are enabled.	
☐	Deploy stage is skipped.	
☐	EventBridge rule is disabled for manual-only release.	

10. Phase 7 - first Dev-to-Prod repository promotion

Step 7.1 - Run the first release with one job

Item	Details
Account	Dev
AWS service	AWS CodePipeline
Objective	Prove cross-account promotion before adding more jobs.
Console navigation	CodePipeline > GlueReleasePipeline1 > Release change
Prerequisites	release-jobs.json contains one tested job.

140. Open release-jobs.json and confirm only one job name is listed.

141. Confirm the listed folder contains JobName.py and JobName.json.

142. Open GlueReleasePipeline1.

143. Choose Release change / Start pipeline execution.

144. Open the execution.

145. Wait for Source to succeed.

146. Open the Build action details and CodeBuild logs.

147. Confirm the log contains Prepared job folder and Release completed successfully.

148. Open the Prod CodeCommit repository.

149. Confirm the one job folder is present and README.md remains.

```
Expected repository result:
<PROD_REPOSITORY_NAME>/
|-- README.md
`-- Demo-copy/
    |-- Demo-copy.py
    `-- Demo-copy.json
```

Step 7.2 - Prove selection control with the second job

Item	Details
Account	Dev and Prod
AWS service	CodeCommit and CodePipeline
Objective	Confirm only manifest entries are promoted.
Console navigation	Edit release-jobs.json and manually run the pipeline
Prerequisites	First job promotion succeeded.

150. Change release-jobs.json so the jobs array contains only the second test job.

151. Commit the manifest.

152. Start the pipeline manually.

153. Confirm the second job folder is added to Prod.

154. Confirm the first job folder remains unchanged.

155. Confirm an unrelated Dev folder was not copied.

Expected result: Prod CodeCommit accumulates Production-approved folders, while each release copies only the manifest entries.

Step 7.3 - Understand no-change behavior

Item	Details
Account	Dev

Item	Details
AWS service	AWS CodePipeline and CodeBuild
Objective	Verify a repeated identical release does not create a meaningless commit.
Console navigation	Run the same manifest without changing the selected folder
Prerequisites	The second release completed.

156. Start the pipeline again without changing the selected job folder.

157. Open CodeBuild logs.

158. Confirm the script reports that no differences were found and nothing was pushed.

159. Confirm the pipeline still succeeds.

Phase 7 completion checklist

Check	Completion criterion	Evidence / value
☐	First approved folder is present in Prod CodeCommit.	
☐	Second approved folder is present after its release.	
☐	Unapproved Dev folders are absent.	
☐	Existing approved folders are not deleted by later releases.	
☐	No-change release succeeds without a new commit.	
☐	Prod commit history identifies the release number and description.	

11. Phase 8 - Production deployment service role

Step 8.1 - Create CodeBuildGlueProductionDeployerRole

Item		Details
Account		Prod
AWS service		AWS IAM
Objective		Create the role used by the manual Production CodeBuild deployment.
Console navigation		IAM > Roles > Create role
Prerequisites		Prod repository, Glue role and assets bucket are ready.
Console field	Select / enter	Why / leave as
Trusted entity type	AWS service	CodeBuild service role.
Use case	CodeBuild	Creates the service trust.
Permissions policies	Do not attach AdministratorAccess	Attach exact inline policy next.
Role name	CodeBuildGlueProductionDeployerRole	Use this in CodeBuild.
Description	Deploys selected Production-approved Glue jobs	Recommended.
Tags	Environment=Prod; Application=Glue-CICD	Recommended.

160. Create the role.

161. Open the role.

162. Copy its ARN.

163. Confirm trust principal `codebuild.amazonaws.com`.

Step 8.2 - Attach the Production deployment inline policy

Item		Details
Account		Prod
AWS service		AWS IAM
Objective		Allow repository read, logs, S3 upload, Glue APIs and PassRole.
Console navigation		IAM > role > Permissions > Create inline policy
Prerequisites		Exact bucket and Glue role values are known.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadProductionRepository",
      "Effect": "Allow",
      "Action": [
        "codecommit:GetRepository",
        "codecommit:GetBranch",
        "codecommit:GitPull"
      ],
      "Resource": "arn:aws:codecommit:<REGION>:<PROD_ACCOUNT_ID>:glue-production"
    },
    {
      "Sid": "WriteCodeBuildLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",

```

```

    "logs:CreateLogStream",
    "logs:PutLogEvents"
  ],
  "Resource": "arn:aws:logs:<REGION>:<PROD_ACCOUNT_ID>:log-group:/aws/codebuild/GlueProductionDeployer*"
},
{
  "Sid": "UploadGlueScripts",
  "Effect": "Allow",
  "Action": [
    "s3:GetBucketLocation",
    "s3:ListBucket"
  ],
  "Resource": "arn:aws:s3:::<PROD_GLUE_ASSETS_BUCKET>"
},
{
  "Sid": "WriteGlueScriptObjects",
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject",
    "s3:AbortMultipartUpload"
  ],
  "Resource": "arn:aws:s3:::<PROD_GLUE_ASSETS_BUCKET>/*"
},
{
  "Sid": "CreateAndUpdateGlueJobs",
  "Effect": "Allow",
  "Action": [
    "glue:GetJob",
    "glue:CreateJob",
    "glue:UpdateJob"
  ],
  "Resource": "*"
},
{
  "Sid": "PassProductionGlueExecutionRole",
  "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::<PROD_ACCOUNT_ID>:role/<PROD_GLUE_ROLE_NAME>",
  "Condition": {
    "StringEquals": {
      "iam:PassedToService": "glue.amazonaws.com"
    }
  }
}
}
}

```

164. Replace <REGION>, <PROD_ACCOUNT_ID>, <PROD_GLUE_ASSETS_BUCKET> and <PROD_GLUE_ROLE_NAME>.

165. Replace glue-production if another repository name is used.

166. Paste the JSON into the inline policy editor.

167. Policy name: GlueProductionDeployerPermissions.

168. Choose Create policy.

169. If the repository or S3 bucket uses a customer KMS key, add the exact KMS actions and key ARN required by your encryption standard.

Why iam:PassRole is required: CreateJob and UpdateJob assign the Production Glue execution role to the job. CodeBuild must be allowed to pass that exact role to glue.amazonaws.com.

Phase 8 completion checklist

Check	Completion criterion	Evidence / value
[]	Production CodeBuild role exists.	
[]	Trust principal is codebuild.amazonaws.com.	
[]	CodeCommit read access is limited to the Prod repository.	
[]	S3 permissions are limited to the Production assets bucket.	
[]	Glue permissions include GetJob, CreateJob	

Check	Completion criterion	Evidence / value
	and UpdateJob.	
[]	iam:PassRole is limited to the exact Production Glue role and glue.amazonaws.com.	
[]	CloudWatch Logs permissions are present.	

12. Phase 9 - Production deployment framework files

Step 9.1 - Create deployment-jobs.txt

Item	Details
Account	Prod
AWS service	AWS CodeCommit
Objective	Choose only the jobs for the current Production deployment.
Console navigation	CodeCommit > Prod repo > Add file > Create file
Prerequisites	At least one approved job folder exists in Prod.

```
# One exact Glue job/folder name per line.
Demo-copy
```

170. Create deployment-jobs.txt in repository root.

171. For the first test, enter one exact job/folder name.

172. Commit message: Add Production deployment list.

This is the deployment control: The Prod repository may contain 200 approved jobs. Only names in deployment-jobs.txt are uploaded and created/updated by the current build.

Step 9.2 - Create glue_config.py

Item	Details
Account	Prod
AWS service	AWS CodeCommit
Objective	Convert Glue Studio JSON to Boto3 CreateJob/UpdateJob input.
Console navigation	CodeCommit > Prod repo > Add file > Create file
Prerequisites	deployment-jobs.txt exists.

173. Create glue_config.py at repository root.

174. Copy the complete file from Appendix B or the ZIP package.

175. Commit message: Add Glue configuration converter.

Important UpdateJob behavior: AWS Glue UpdateJob completely overwrites the previous job definition and unspecified configuration is removed or reset. The converter therefore includes common supported fields, not only the four overridden fields. Reference AWS-9.

Step 9.3 - Create deploy.py

Item	Details
Account	Prod
AWS service	AWS CodeCommit
Objective	Upload selected scripts and create/update selected Glue jobs.
Console navigation	CodeCommit > Prod repo > Add file > Create file
Prerequisites	glue_config.py exists.

176. Create deploy.py at repository root.

177. Copy the complete file from Appendix B or ZIP package.

178. Commit message: Add selected Glue job deployer.

Step 9.4 - Create the Production buildspec.yml

Item	Details
Account	Prod
AWS service	AWS CodeCommit
Objective	Run syntax checks and deploy.py.
Console navigation	CodeCommit > Prod repo > Add file > Create file
Prerequisites	deploy.py and glue_config.py exist.

```
version: 0.2

phases:
  install:
    runtime-versions:
      python: 3.11
  pre_build:
    commands:
      - echo "Validating Production deployment files"
      - python3 -m py_compile deploy.py glue_config.py
      - test -f deployment-jobs.txt
  build:
    commands:
      - echo "Jobs selected for Production deployment:"
      - cat deployment-jobs.txt
      - python3 deploy.py
```

179. Create buildspec.yml at repository root.

180. Paste YAML using spaces only.

181. Commit message: Add Production deployment buildspec.

Step 9.5 - Review the final Prod repository structure

Item	Details
Account	Prod
AWS service	AWS CodeCommit
Objective	Confirm CodeBuild will receive job folders and framework files.
Console navigation	CodeCommit > Prod repository > main
Prerequisites	All four files are committed.

```
<PROD_REPOSITORY_NAME>/
|-- README.md
|-- Demo-copy/
|   |-- Demo-copy.py
|   `-- Demo-copy.json
|-- SecondJob/
|   |-- SecondJob.py
|   `-- SecondJob.json
|-- deployment-jobs.txt
|-- glue_config.py
|-- deploy.py
`-- buildspec.yml
```

Phase 9 completion checklist

Check	Completion criterion	Evidence / value
[]	deployment-jobs.txt contains one exact test job.	
[]	glue_config.py exists at root.	

Check	Completion criterion	Evidence / value
☐	deploy.py exists at root.	
☐	buildspec.yml exists at root.	
☐	All files are committed to main.	
☐	No Dev credentials or Dev role ARN are embedded in Prod scripts.	

13. Phase 10 - Production CodeBuild project

Step 10.1 - Create GlueProductionDeployer

Item	Details
Account	Prod
AWS service	AWS CodeBuild
Objective	Create a manually started CodeBuild project that reads Prod CodeCommit directly.
Console navigation	CodeBuild > Build projects > Create build project
Prerequisites	Production role and repository framework are complete.

Project configuration

Console field	Select / enter	Why / leave as
Project name	GlueProductionDeployer	Exact name used in IAM log resource.
Description	Deploys selected Production-approved Glue jobs	Recommended.
Build badge	Disabled	Not required.
Concurrent build limit	1	Prevents overlapping updates to the same jobs.
Tags	Environment=Prod; Application=Glue-CICD	Recommended.

Source

Console field	Select / enter	Why / leave as
Source provider	AWS CodeCommit	This is a standalone manual CodeBuild project.
Repository	<PROD_REPOSITORY_NAME>	Select glue-production or approved name.
Reference type / Source version	Branch: main	Deploy only the Production branch.
Git clone depth	1	Only current files are needed.
Git submodules	Disabled	No submodules are used.
Primary source webhook	Disabled	Production deployment is manual.
Report build status	Disabled	Not required for CodeCommit source.

Why CodeCommit is correct here: The Production deployment project is created directly in CodeBuild and is manually started, so its source provider must be AWS CodeCommit. CodePipeline is not used for this Production step.

Environment

Console field	Select / enter	Why / leave as
Provisioning model	On-demand	No reserved fleet cost.
Environment image	Managed image	AWS-maintained.
Compute	EC2	Supports the workflow.
Operating system	Amazon Linux	Current managed Amazon Linux.

Console field	Select / enter	Why / leave as
Runtime(s)	Standard	Includes Python, AWS CLI and Git.
Image	aws/codebuild/amazonlinux-x86_64-standard:5.0 or current approved equivalent	Use latest revision in runtime family.
Image version	Always use latest for runtime version	Patched revisions.
Environment type	Linux EC2	Default x86_64.
Compute type	Small / BUILD_GENERAL1_SMALL	Sufficient for selected deployments.
Service role	Existing service role	Use Phase 8 role.
Role ARN	arn:aws:iam::<PROD_ACCOUNT_ID>:role/CodeBuildGlueProductionDeployerRole	Exact role.
Privileged mode	Disabled	No Docker.
VPC	No VPC	Only AWS public APIs are called; add VPC only when endpoints/policy require.
Auto-retry limit	0	Review failures before retrying.
Timeout	60 minutes	Increase only for very large batches.
Queued timeout	Default	No change.

Environment variables

Name	Value	Type
PROD_GLUE_ROLE_ARN	arn:aws:iam::<PROD_ACCOUNT_ID>:role/<PROD_GLUE_ROLE_NAME>	Plaintext
PROD_GLUE_ASSETS_BUCKET	<PROD_GLUE_ASSETS_BUCKET>	Plaintext

Buildspec, artifacts, cache and logs

Console field	Select / enter	Why / leave as
Build specifications	Use a buildspec file	Reads root buildspec.yml.
Buildspec name	Leave blank or buildspec.yml	Default root file.
Batch configuration	Disabled	One controlled deployment build.
Artifacts type	No artifacts	The build changes S3 and Glue directly.
Cache type	No cache	Not needed.
CloudWatch logs	Enabled	Required for evidence.
Log group	/aws/codebuild/GlueProductionDeployer or generated	Use explicit name when required.
S3 logs	Disabled	Optional under policy.
Encryption key	Default	No build artifacts are produced.

182. Choose Create build project.

183. Open the created project.

184. Choose Edit > Source and confirm AWS CodeCommit / main.

185. Choose Edit > Environment and confirm the exact existing role and two environment variables.

Expected result: GlueProductionDeployer is a standalone manual build project with CodeCommit source and no artifacts.

Phase 10 completion checklist

Check	Completion criterion	Evidence / value
☐	Project GlueProductionDeployer exists in Prod.	
☐	Source provider is AWS CodeCommit.	
☐	Repository and main branch are correct.	
☐	Webhook is disabled.	
☐	Compute is Small and privileged mode is disabled.	
☐	Existing Production deployer role is selected.	
☐	PROD_GLUE_ROLE_ARN is exact.	
☐	PROD_GLUE_ASSETS_BUCKET contains only the bucket name.	
☐	Buildspec is root buildspec.yml.	
☐	Artifacts and cache are disabled.	
☐	CloudWatch logs are enabled.	
☐	Concurrent build limit is 1.	

14. Phase 11 - first Production Glue deployment

Step 11.1 - Run the first one-job deployment

Item	Details
Account	Prod
AWS service	AWS CodeBuild
Objective	Create or update one selected Glue job.
Console navigation	CodeBuild > GlueProductionDeployer > Start build
Prerequisites	deployment-jobs.txt contains one approved job.

186. Open deployment-jobs.txt and confirm only one exact job name is active.

187. Confirm the matching folder contains .py and .json.

188. Open GlueProductionDeployer.

189. Choose Start build.

190. Do not override source version or environment variables.

191. Choose Start build.

192. Open Phase details and Tail logs.

193. Confirm validation passes.

194. Confirm the script upload message.

195. Confirm CREATED: JobName or UPDATED: JobName.

196. Confirm the build status is Succeeded.

```
Expected log pattern:
Jobs selected for Production deployment:
Demo-copy
Uploading Demo-copy/Demo-copy.py to s3://<PROD_GLUE_ASSETS_BUCKET>/scripts/Demo-copy.py
CREATED: Demo-copy
Production deployment completed for 1 job(s).
```

Step 11.2 - Validate the Production S3 script

Item	Details
Account	Prod
AWS service	Amazon S3
Objective	Confirm the selected script was uploaded to the correct location.
Console navigation	S3 > <PROD_GLUE_ASSETS_BUCKET> > scripts
Prerequisites	Build succeeded.

Check	Completion criterion	Evidence / value
[]	scripts/<JobName>.py exists.	
[]	Object Last modified time matches the deployment.	
[]	Bucket Region is correct.	
[]	Object encryption is correct.	
[]	No script was uploaded to the Dev bucket.	
[]	Only selected deployment job scripts changed.	

Step 11.3 - Validate the Production Glue job configuration

Item	Details
Account	Prod
AWS service	AWS Glue
Objective	Confirm the four Production overrides and preserved settings.
Console navigation	Glue > ETL jobs > select JobName > Job details
Prerequisites	Build succeeded and script exists.
Validation point	Expected value
IAM Role	<PROD_GLUE_ROLE_ARN>
Script path	s3://<PROD_GLUE_ASSETS_BUCKET>/scripts/<JobName>.py
--spark-event-logs-path	s3://<PROD_GLUE_ASSETS_BUCKET>/sparkHistoryLogs/
--TempDir	s3://<PROD_GLUE_ASSETS_BUCKET>/temporary/
Glue version	Matches the job JSON.
Worker type and workers	Match the job JSON.
Timeout and retries	Match the job JSON.
Default arguments other than two overridden paths	Match the job JSON.
Command name	glueetl for Spark ETL, or the correct job type.
Connections and security configuration	Match JSON and exist in Prod.
Job mode / visual graph	Preserved when present in JSON.
glueetl is correct: For an Apache Spark ETL job, AWS Glue JobCommand.Name must be glueetl. It is not a misspelling. Reference AWS-10.	

Step 11.4 - Run the deployed Glue job safely

Item	Details
Account	Prod
AWS service	AWS Glue and CloudWatch Logs
Objective	Prove the job can access all Production dependencies.
Console navigation	Glue > ETL jobs > Run
Prerequisites	The change owner has approved a test run and safe input range.

197. Confirm the run will not duplicate or overwrite Production business data.

198. Use approved test arguments or a non-destructive input where available.

199. Choose Run.

200. Monitor Run details and CloudWatch logs.

201. Confirm connections, secrets, Catalog, S3 and network access.

202. Record input counts, output counts and run ID.

203. If the job fails, distinguish deployment configuration errors from data/network dependency errors.

Step 11.5 - Test update behavior

Item	Details
Account	Prod
AWS service	AWS Glue, CodeCommit and CodeBuild
Objective	Prove an existing job is updated rather than duplicated.
Console navigation	Change a harmless approved job property in Dev and repeat promotion/deployment
Prerequisites	Initial job creation succeeded.

204. Change a safe property such as description in Dev Glue Studio.

205. Push the job to Dev CodeCommit.

206. List that job in release-jobs.json and run Dev pipeline.

207. Confirm the Prod repository commit changed only that folder.

208. List the job in deployment-jobs.txt.

209. Run Production CodeBuild.

210. Confirm log shows UPDATED.

211. Confirm the same Glue job name exists once and contains the new property.

Phase 11 completion checklist

Check	Completion criterion	Evidence / value
☐	Production build succeeded.	
☐	Selected script exists in Production S3.	
☐	Glue job exists once.	
☐	Production role is correct.	
☐	ScriptLocation is correct.	
☐	Spark event-log and TempDir paths are correct.	
☐	Worker, version, retries, timeout and other arguments match JSON.	
☐	Dependencies exist and runtime test succeeded.	
☐	Second deployment updated the job rather than creating a duplicate.	
☐	Evidence is attached to the change record.	

15. Phase 12 - normal release, hotfix and rollback procedures

12.1 Normal release procedure

Sequence	Owner action	Control file
1	Test and save Glue jobs in Dev.	None
2	Push each approved job from Glue Studio to Dev CodeCommit.	None
3	Edit release-jobs.json to list only approved folders. Increment release version.	release-jobs.json
4	Commit the release manifest.	release-jobs.json
5	Manually start GlueReleasePipeline1.	release-jobs.json
6	Review Prod CodeCommit commit and folder diff.	Prod repository history
7	Edit deployment-jobs.txt to list only jobs approved for the Prod change.	deployment-jobs.txt
8	Commit deployment list.	deployment-jobs.txt
9	Manually start GlueProductionDeployer.	deployment-jobs.txt
10	Validate S3, Glue configuration and runtime results.	Change evidence

Example initial migration:

release-jobs.json → 200 approved folders copied to Prod repository

deployment-jobs.txt → deploy in controlled batches, for example 25 jobs per build

Example later release:

release-jobs.json → 5 changed folders

deployment-jobs.txt → the same 5 jobs, or a smaller approved subset

12.2 Hotfix procedure

212. Create or update the hotfix in Dev and test it.
213. Push only the hotfix job folder to Dev CodeCommit.
214. Create a release manifest containing only the hotfix job.
215. Run the Dev pipeline manually.
216. Review the exact Prod repository diff.
217. Set deployment-jobs.txt to the one hotfix job.
218. Run Production CodeBuild manually.
219. Validate and document the hotfix.
220. After completion, remove stale job names from deployment-jobs.txt or replace it in the next change.

12.3 Rollback one job

221. Open Prod CodeCommit and identify the last known-good commit for the job folder.
222. Restore JobName.py and JobName.json from that commit into main using Git or a controlled revert commit.
223. Set deployment-jobs.txt to only that job name.
224. Commit the rollback.
225. Start GlueProductionDeployer.
226. Confirm UPDATED in logs.
227. Validate the restored script path and job properties.
228. Run a safe validation job run.

12.4 Rollback multiple jobs or an entire release

- 229. Identify the Production repository commit before the failed release.
- 230. Create a revert commit for every affected folder; do not delete unrelated older approved folders.
- 231. List all reverted jobs in deployment-jobs.txt.
- 232. Deploy in batches if the release is large.
- 233. Validate each batch and record results.
- 234. Do not roll back data outputs automatically unless a separate data-recovery plan exists.

Code rollback is not data rollback: Reverting a Glue script/job definition does not undo files, database writes, bookmarks, messages or downstream updates produced by an earlier job run. Data rollback must be designed separately.

16. Troubleshooting by error message

Error / symptom	Where to check	Cause	Exact corrective action
AssumeRole AccessDenied	Dev CodeBuild logs; both IAM roles	Dev role policy or Prod trust Principal does not match.	Copy both role ARNs, correct the Dev sts:AssumeRole Resource and Prod trust Principal, then rerun.
403 during git clone or push	Dev CodeBuild logs; Prod release role policy	Temporary Prod role lacks GitPull/GitPush or repository ARN is wrong.	Correct Region/account/repository in Prod inline policy; confirm credential helper uses assumed-role environment.
fatal: HOME not set	Dev CodeBuild logs	Git cannot write global configuration.	release.py sets HOME=/tmp. Confirm the current script version is committed.
Repository branch main not found	Prod CodeCommit	Repo has no initial commit or wrong branch.	Create README.md on main or correct PROD_BRANCH.
Missing JobName.py / JobName.json	Dev or Prod CodeCommit	Folder/file names do not match exact job name.	Rename folder and both files or correct the control list entry.
No differences were found	Dev CodeBuild logs	Selected Prod folder already matches Dev.	Expected; no action unless a change was expected. Compare commits and confirm Glue Studio push occurred.
CodeBuild source does not show CodePipeline	CodeBuild create screen	Project was created directly in CodeBuild console.	Create/select the project from the CodePipeline Build stage. This is expected behavior per AWS-5.
CodePipeline starts on every commit	EventBridge rules	Console-created CodeCommit rule is enabled.	Disable the rule targeting GlueReleasePipeline1 and start releases manually.
CodePipeline Source succeeds, Build cannot find buildspec	Dev repository root	buildspec.yml missing or path wrong.	Commit buildspec.yml at root or set Buildspec name to its relative path.
YAML_FILE_ERROR	CodeBuild logs	Tabs or invalid indentation in buildspec.	Replace tabs with spaces and validate YAML.
CodeCommit GitPull denied in Prod CodeBuild	Prod CodeBuild role	Role lacks read permission for source repository.	Add GetRepository, GetBranch and GitPull to exact Prod repo ARN.
s3:PutObject AccessDenied	Prod CodeBuild logs	Bucket or KMS permissions are missing.	Correct bucket ARN; add PutObject/AbortMultipartUpload and required KMS permissions.
iam:PassRole AccessDenied	Prod CodeBuild logs	PassRole resource or service condition is wrong.	Allow exact Prod Glue role and iam:PassedToService=glue.amazonaws.com.
glue:CreateJob or UpdateJob denied	Prod CodeBuild role	Glue API actions are missing or blocked by SCP.	Add GetJob/CreateJob/UpdateJob and resolve SCP or permission boundary.
InvalidInputException: MaxCapacity	Prod CodeBuild logs	MaxCapacity submitted with WorkerType/NumberOfWorkers.	Use the provided glue_config.py; it submits one capacity model only.
InvalidInputException: ExecutionClass	Prod CodeBuild logs	FLEX used with unsupported Glue version or command.	Use STANDARD or update job version/type according to Glue support.
Job settings disappeared after update	Glue job details	UpdateJob overwrites unspecified properties.	Use the complete converter, restore from Prod repository, and redeploy.
Job created but runtime fails connection not found	Glue run logs	Prod dependency is missing or differently named.	Create/rename the Prod Glue connection or map the configuration before deployment.
Job runtime AccessDenied to S3/KMS/Secrets	Glue run logs; Glue role	Production Glue execution role lacks data access.	Update the Glue execution role, not the CodeBuild role.
Wrong Dev paths remain in default arguments	Glue job details	Only TempDir/event logs are automatically overridden; another Dev path is job-specific.	Change the Dev job configuration before release or extend controlled environment mapping.
Build processes jobs not intended for	Control files	Extra names remain in release-	Replace the file with only current-

Error / symptom	Where to check	Cause	Exact corrective action
release		jobs.json or deployment-jobs.txt.	change names and commit before running.
Concurrent modification / Git conflict	CodeBuild logs	Two executions overlapped or someone edited Prod main during release.	Keep concurrency limit 1, Queued mode, resolve Git conflict and rerun.
CloudWatch logs missing	CodeBuild project/role	Logs disabled or role lacks logs actions.	Enable CloudWatch logs and correct log-group ARN in role policy.

17. Security, dependencies, limitations and scaling

17.1 Security controls

- Use exact role ARNs in cross-account trust; do not trust the entire Dev account unless formally approved.
- Scope CodeCommit permissions to one Production repository.
- Scope iam:PassRole to one Production Glue role and glue.amazonaws.com.
- Keep both Dev release and Production deployment manually started when change approval is required.
- Enable CloudTrail and retain CodeBuild/CodePipeline logs for audit evidence.
- Enable S3 versioning for Production scripts.
- Do not store access keys, passwords, database credentials or tokens in repository files or Glue default arguments.
- Use Secrets Manager, SSM Parameter Store or Glue connections for secrets.
- Apply branch review/approval controls to Production main where supported by your governance process.

17.2 Scaling to 200 or more jobs

- Release files list exact names; the scripts do not scan or mirror all Dev jobs.
- For the first migration, promote approved folders in logical groups rather than all 200 at once.
- Deploy in batches such as 10-25 jobs so failed dependencies are easier to identify.
- Use job families or application owners to create release batches.
- Keep CodeBuild concurrent limit at 1 for a single Production branch.
- After stable operation, add a validation script that checks all dependency names before Glue updates.
- For multiple Glue execution roles, add an approved mapping file rather than permitting arbitrary role ARNs.

17.3 Known limitations

- The solution creates/updates jobs; it does not migrate Glue connections, crawlers, workflows, triggers, schedules or Catalog objects.
- Only the two environment path arguments are automatically rewritten. Other Dev-specific arguments must be designed for environment portability.
- SourceControlDetails is not copied to the Production Glue job.
- Glue tags require separate tag management if required.
- Job runs are not started automatically by the deployment script.
- A successful API deployment does not prove runtime dependency access.
- Visual job graph compatibility depends on codeGenConfigurationNodes present in the Glue Studio JSON.

17.4 Earlier S3 staging approach used during the proof of concept

POC component	What was done	Use in final implementation
Prod artifact bucket	prod-glue-deployment-artifacts with release/job prefixes	Not required by the final direct CodeCommit promotion path.
Dev S3 uploader role	CodeBuildGlueArtifactUploaderRole	Not required by release.py direct Git push.
Cross-account bucket policy	Allowed Dev CodeBuild to upload artifacts	Remove when no other process needs it.
Why changed	The staged approach proved cross-account transfer but added another copy and approval surface.	Prod CodeCommit now provides version history and controlled source before deployment.
Implementation decision: For new accounts, build the final CodeCommit-to-CodeCommit path first. Recreate the S3 staging method only when a formal artifact repository is a stated requirement.		

18. Appendix - complete scripts, IAM policies and AWS references

Appendix A - Dev repository files

A.1 release-jobs.json

```
{
  "release": "1.0.0",
  "description": "Initial Production release",
  "jobs": [
    "Demo-copy"
  ]
}
```

A.2 buildspec.yml

```
version: 0.2

phases:
  install:
    runtime-versions:
      python: 3.11
  pre_build:
    commands:
      - echo "Validating release files"
      - python3 -m py_compile release.py
      - test -f release-jobs.json
  build:
    commands:
      - echo "Starting selective Glue release to the Production repository"
      - python3 release.py
```

A.3 release.py

```
#!/usr/bin/env python3
"""Promote selected Glue job folders from Dev to the Prod CodeCommit repo."""

from __future__ import annotations

import json
import os
import shutil
import subprocess
import sys
from pathlib import Path
from typing import Sequence

import boto3

REGION = os.environ.get("AWS_REGION") or os.environ.get("AWS_DEFAULT_REGION", "us-east-1")
PROD_ROLE_ARN = os.environ["PROD_RELEASE_ROLE_ARN"]
PROD_REPOSITORY_NAME = os.environ.get("PROD_REPOSITORY_NAME", "glue-production")
PROD_BRANCH = os.environ.get("PROD_BRANCH", "main")
MANIFEST_FILE = Path(os.environ.get("RELEASE_MANIFEST", "release-jobs.json"))
WORK_DIR = Path("/tmp/glue-production")
REPO_URL = (
    f"https://git-codecommit.{REGION}.amazonaws.com/v1/repos/"
    f"{PROD_REPOSITORY_NAME}"
)

def run(
    command: Sequence[str],
    *,
    cwd: Path | None = None,
    env: dict[str, str] | None = None,
    check: bool = True,
) -> subprocess.CompletedProcess[str]:
    """Run a command and print it without exposing credentials."""
    print("$", " ".join(command), flush=True)
    result = subprocess.run(
        list(command),
        cwd=str(cwd) if cwd else None,
        env=env,
        text=True,
        check=False,
    )
    if check and result.returncode != 0:
        raise RuntimeError(
```

```

        f"Command failed with exit code {result.returncode}: {' '.join(command)}"
    )
    return result

def load_manifest() -> tuple[str, str, list[str]]:
    if not MANIFEST_FILE.exists():
        raise FileNotFoundError(f"Release manifest not found: {MANIFEST_FILE}")

    manifest = json.loads(MANIFEST_FILE.read_text(encoding="utf-8"))
    release = str(manifest.get("release", "unversioned"))
    description = str(manifest.get("description", "")).strip()
    jobs = manifest.get("jobs")

    if not isinstance(jobs, list) or not jobs:
        raise ValueError("release-jobs.json must contain a non-empty 'jobs' array")

    clean_jobs: list[str] = []
    seen: set[str] = set()
    for raw_job in jobs:
        job = str(raw_job).strip()
        if not job or job.startswith("#"):
            continue
        if job in seen:
            raise ValueError(f"Duplicate job in release manifest: {job}")
        seen.add(job)
        clean_jobs.append(job)

    if not clean_jobs:
        raise ValueError("No valid job names were found in release-jobs.json")

    return release, description, clean_jobs

def validate_job_folder(job: str) -> None:
    folder = Path(job)
    required = [folder, folder / f"{job}.py", folder / f"{job}.json"]
    missing = [str(item) for item in required if not item.exists()]
    if missing:
        raise FileNotFoundError(
            f"Job '{job}' is incomplete. Missing: {' '.join(missing)}"
        )

def assumed_role_environment() -> dict[str, str]:
    credentials = boto3.client("sts", region_name=REGION).assume_role(
        RoleArn=PROD_ROLE_ARN,
        RoleSessionName="GlueReleaseCodeBuild",
    )["Credentials"]

    environment = os.environ.copy()
    environment.update(
        {
            "HOME": "/tmp",
            "AWS_REGION": REGION,
            "AWS_DEFAULT_REGION": REGION,
            "AWS_ACCESS_KEY_ID": credentials["AccessKeyId"],
            "AWS_SECRET_ACCESS_KEY": credentials["SecretAccessKey"],
            "AWS_SESSION_TOKEN": credentials["SessionToken"],
        }
    )
    return environment

def configure_git(environment: dict[str, str]) -> None:
    commands = [
        ["git", "config", "--global", "credential.helper", "!aws codecommit credential-helper $@"],
        ["git", "config", "--global", "credential.UseHttpPath", "true"],
        ["git", "config", "--global", "user.name", "Glue Release Pipeline"],
        ["git", "config", "--global", "user.email", "glue-release@example.invalid"],
        ["git", "config", "--global", "init.defaultBranch", PROD_BRANCH],
    ]
    for command in commands:
        run(command, env=environment)

def prepare_repository(environment: dict[str, str]) -> None:
    shutil.rmtree(WORK_DIR, ignore_errors=True)
    run(["git", "clone", REPO_URL, str(WORK_DIR)], env=environment)

    remote_branch = run(
        ["git", "ls-remote", "--exit-code", "--heads", "origin", PROD_BRANCH],
        cwd=WORK_DIR,
        env=environment,
    )

```

```

        check=False,
    )

    if remote_branch.returncode == 0:
        run(["git", "fetch", "origin", PROD_BRANCH], cwd=WORK_DIR, env=environment)
        run(
            ["git", "checkout", "-B", PROD_BRANCH, f'origin/{PROD_BRANCH}'],
            cwd=WORK_DIR,
            env=environment,
        )
    else:
        run(["git", "checkout", "-B", PROD_BRANCH], cwd=WORK_DIR, env=environment)

def copy_jobs(jobs: list[str]) -> None:
    for job in jobs:
        target = WORK_DIR / job
        shutil.rmtree(target, ignore_errors=True)
        shutil.copytree(Path(job), target)
        print(f'Prepared job folder: {job}', flush=True)

def commit_and_push(
    release: str,
    description: str,
    environment: dict[str, str],
) -> None:
    run(["git", "add", "--all"], cwd=WORK_DIR, env=environment)
    unchanged = run(
        ["git", "diff", "--cached", "--quiet"],
        cwd=WORK_DIR,
        env=environment,
        check=False,
    )
    if unchanged.returncode == 0:
        print("No differences were found. Nothing was pushed.", flush=True)
        return

    message = f'Glue release {release}'
    if description:
        message += f' - {description}'

    run(["git", "commit", "-m", message], cwd=WORK_DIR, env=environment)
    run(
        ["git", "push", "-u", "origin", f'HEAD:{PROD_BRANCH}'],
        cwd=WORK_DIR,
        env=environment,
    )

def main() -> int:
    release, description, jobs = load_manifest()
    for job in jobs:
        validate_job_folder(job)

    environment = assumed_role_environment()
    configure_git(environment)
    prepare_repository(environment)
    copy_jobs(jobs)
    commit_and_push(release, description, environment)
    shutil.rmtree(WORK_DIR, ignore_errors=True)
    print(f'Release {release} completed successfully.', flush=True)
    return 0

if __name__ == "__main__":
    try:
        raise SystemExit(main())
    except Exception as exc: # CodeBuild should fail with a clear message.
        print(f'ERROR: {exc}', file=sys.stderr, flush=True)
        raise

```

Appendix B - Production repository files

B.1 deployment-jobs.txt

```
# One exact Glue job/folder name per line.
Demo-copy
```

B.2 buildspec.yml

```
version: 0.2

phases:
  install:
    runtime-versions:
      python: 3.11
  pre_build:
    commands:
      - echo "Validating Production deployment files"
      - python3 -m py_compile deploy.py glue_config.py
      - test -f deployment-jobs.txt
  build:
    commands:
      - echo "Jobs selected for Production deployment:"
      - cat deployment-jobs.txt
      - python3 deploy.py
```

B.3 glue_config.py

```
"""Convert AWS Glue Studio source-control JSON into boto3 JobInput fields."""

from __future__ import annotations

import copy
import json
import os
from typing import Any

PROD_ROLE_ARN = os.environ["PROD_GLUE_ROLE_ARN"]
PROD_BUCKET = os.environ["PROD_GLUE_ASSETS_BUCKET"]

def _copy_if_present(
    destination: dict[str, Any],
    source: dict[str, Any],
    source_key: str,
    destination_key: str,
) -> None:
    value = source.get(source_key)
    if value is not None:
        destination[destination_key] = copy.deepcopy(value)

def _normalise_connections(value: Any) -> dict[str, list[str]] | None:
    if value is None:
        return None
    if isinstance(value, list):
        return {"Connections": [str(item) for item in value]}
    if isinstance(value, dict):
        connections = value.get("connections", value.get("Connections"))
        if connections is not None:
            return {"Connections": [str(item) for item in connections]}
        raise ValueError(f"Unsupported connections format: {value!r}")

def _normalise_codegen_nodes(value: Any) -> dict[str, Any] | None:
    if value is None:
        return None
    if isinstance(value, dict):
        return copy.deepcopy(value)
    if isinstance(value, str):
        parsed = json.loads(value)
        if not isinstance(parsed, dict):
            raise ValueError("codegenConfigurationNodes must decode to an object")
        return parsed
    raise ValueError("Unsupported codeGenConfigurationNodes format")

def create_job_input(source: dict[str, Any], job_name: str) -> dict[str, Any]:
    """Build CreateJob/UpdateJob input and override Production-only values."""
    command_source = source.get("command") or {}
    if not command_source.get("name"):
```

```

raise ValueError(f"{job_name}: command.name is missing from the JSON")

command: dict[str, Any] = {
    "Name": command_source["name"],
    "ScriptLocation": f"s3://{PROD_BUCKET}/scripts/{job_name}.py",
}
_copy_if_present(command, command_source, "pythonVersion", "PythonVersion")
_copy_if_present(command, command_source, "runtime", "Runtime")

default_arguments = copy.deepcopy(source.get("defaultArguments") or {})
default_arguments["--spark-event-logs-path"] = (
    f"s3://{PROD_BUCKET}/sparkHistoryLogs/"
)
default_arguments["--TempDir"] = f"s3://{PROD_BUCKET}/temporary/"

job_input: dict[str, Any] = {
    "Role": PROD_ROLE_ARN,
    "Command": command,
    "DefaultArguments": default_arguments,
}

direct_fields = {
    "description": "Description",
    "maxRetries": "MaxRetries",
    "timeout": "Timeout",
    "glueVersion": "GlueVersion",
    "executionClass": "ExecutionClass",
    "securityConfiguration": "SecurityConfiguration",
    "jobMode": "JobMode",
    "jobRunQueueingEnabled": "JobRunQueueingEnabled",
    "maintenanceWindow": "MaintenanceWindow",
    "nonOverridableArguments": "NonOverridableArguments",
}
for source_key, destination_key in direct_fields.items():
    _copy_if_present(job_input, source, source_key, destination_key)

execution_property = source.get("executionProperty")
if execution_property and execution_property.get("maxConcurrentRuns") is not None:
    job_input["ExecutionProperty"] = {
        "MaxConcurrentRuns": execution_property["maxConcurrentRuns"]
    }

notification_property = source.get("notificationProperty")
if notification_property:
    delay = notification_property.get(
        "notifyDelayAfter", notification_property.get("NotifyDelayAfter")
    )
    if delay is not None:
        job_input["NotificationProperty"] = {"NotifyDelayAfter": delay}

connections = _normalise_connections(source.get("connections"))
if connections:
    job_input["Connections"] = connections

codegen_nodes = _normalise_codegen_nodes(source.get("codegenConfigurationNodes"))
if codegen_nodes:
    job_input["CodeGenConfigurationNodes"] = codegen_nodes

worker_type = source.get("workerType")
number_of_workers = source.get("numberOfWorkers")
max_capacity = source.get("maxCapacity")

if worker_type is not None or number_of_workers is not None:
    if worker_type is None or number_of_workers is None:
        raise ValueError(
            f"{job_name}: workerType and numberOfWorkers must be supplied together"
        )
    job_input["WorkerType"] = worker_type
    job_input["NumberOfWorkers"] = number_of_workers
elif max_capacity is not None:
    job_input["MaxCapacity"] = max_capacity
elif source.get("allocatedCapacity") is not None:
    # allocatedCapacity is deprecated; use it only as a legacy fallback.
    job_input["MaxCapacity"] = source["allocatedCapacity"]

# SourceControlDetails is intentionally not copied. The Production repository
# and deployment pipeline are the source of truth for Production.
return job_input

```

B.4 deploy.py

```

#!/usr/bin/env python3
"""Create or update only the Glue jobs listed in deployment-jobs.txt."""

```

```

from __future__ import annotations

import json
import os
import sys
from pathlib import Path

import boto3
from botocore.exceptions import ClientError

from glue_config import PROD_BUCKET, create_job_input

REGION = os.environ.get("AWS_REGION") or os.environ.get("AWS_DEFAULT_REGION", "us-east-1")
DEPLOYMENT_LIST = Path(os.environ.get("DEPLOYMENT_LIST", "deployment-jobs.txt"))

s3 = boto3.client("s3", region_name=REGION)
glue = boto3.client("glue", region_name=REGION)

def read_jobs() -> list[str]:
    if not DEPLOYMENT_LIST.exists():
        raise FileNotFoundError(f"Deployment list not found: {DEPLOYMENT_LIST}")

    jobs: list[str] = []
    seen: set[str] = set()
    for raw_line in DEPLOYMENT_LIST.read_text(encoding="utf-8").splitlines():
        job = raw_line.strip()
        if not job or job.startswith("#"):
            continue
        if job in seen:
            raise ValueError(f"Duplicate job in deployment list: {job}")
        seen.add(job)
        jobs.append(job)

    if not jobs:
        raise ValueError("No deployable jobs were found in deployment-jobs.txt")
    return jobs

def validate_files(job: str) -> tuple[Path, Path]:
    script = Path(job) / f"{job}.py"
    definition = Path(job) / f"{job}.json"
    missing = [str(path) for path in (script, definition) if not path.exists()]
    if missing:
        raise FileNotFoundError(f"{job}: missing {', '.join(missing)}")
    return script, definition

def job_exists(job: str) -> bool:
    try:
        glue.get_job(JobName=job)
        return True
    except glue.exceptions.EntityNotFoundException:
        return False

def deploy_job(job: str) -> None:
    script, definition = validate_files(job)
    source = json.loads(definition.read_text(encoding="utf-8"))
    job_input = create_job_input(source, job)

    key = f"scripts/{job}.py"
    print(f"Uploading {script} to s3://{PROD_BUCKET}/{key}", flush=True)
    s3.upload_file(str(script), PROD_BUCKET, key)

    if job_exists(job):
        glue.update_job(JobName=job, JobUpdate=job_input)
        print(f"UPDATED: {job}", flush=True)
    else:
        glue.create_job(Name=job, **job_input)
        print(f"CREATED: {job}", flush=True)

def main() -> int:
    jobs = read_jobs()
    failures: list[str] = []

    for job in jobs:
        try:
            deploy_job(job)
        except (ClientError, ValueError, FileNotFoundError, json.JSONDecodeError) as exc:
            failures.append(job)
            print(f"FAILED: {job}: {exc}", file=sys.stderr, flush=True)

```

```
if failures:
    raise RuntimeError(f"Deployment failed for: {' '.join(failures)}")

print(f"Production deployment completed for {len(jobs)} job(s).", flush=True)
return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

Appendix C - IAM policies

C.1 Dev CodeBuild trust policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "codebuild.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

C.2 Dev CodeBuild permissions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "WriteCodeBuildLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:<REGION>:<DEV_ACCOUNT_ID>:log-group:/aws/codebuild/GlueReleaseBuilder*"
    },
    {
      "Sid": "AssumeProductionReleaseRole",
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::<PROD_ACCOUNT_ID>:role/GlueReleasePushRole"
    }
  ]
}
```

C.3 Prod release role trust

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "TrustDevCodeBuildRole",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<DEV_ACCOUNT_ID>:role/service-role/GlueReleaseBuilderRole"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

C.4 Prod release role permissions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadAndPushProductionRepository",
      "Effect": "Allow",
      "Action": [
        "codecommit:GetRepository",
        "codecommit:GetBranch",
        "codecommit:GitPull",
        "codecommit:GitPush"
      ],
      "Resource": "arn:aws:codecommit:<REGION>:<PROD_ACCOUNT_ID>:glue-production"
    }
  ]
}
```

C.5 Prod CodeBuild trust

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```

"Effect": "Allow",
"Principal": {
  "Service": "codebuild.amazonaws.com"
},
"Action": "sts:AssumeRole"
}
}
}

```

C.6 Prod CodeBuild permissions

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadProductionRepository",
      "Effect": "Allow",
      "Action": [
        "codecommit:GetRepository",
        "codecommit:GetBranch",
        "codecommit:GitPull"
      ],
      "Resource": "arn:aws:codecommit:<REGION>:<PROD_ACCOUNT_ID>:glue-production"
    },
    {
      "Sid": "WriteCodeBuildLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:<REGION>:<PROD_ACCOUNT_ID>:log-group:/aws/codebuild/GlueProductionDeployer*"
    },
    {
      "Sid": "UploadGlueScripts",
      "Effect": "Allow",
      "Action": [
        "s3:GetBucketLocation",
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::<PROD_GLUE_ASSETS_BUCKET>"
    },
    {
      "Sid": "WriteGlueScriptObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:AbortMultipartUpload"
      ],
      "Resource": "arn:aws:s3:::<PROD_GLUE_ASSETS_BUCKET>/*"
    },
    {
      "Sid": "CreateAndUpdateGlueJobs",
      "Effect": "Allow",
      "Action": [
        "glue:GetJob",
        "glue:CreateJob",
        "glue:UpdateJob"
      ],
      "Resource": "*"
    },
    {
      "Sid": "PassProductionGlueExecutionRole",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::<PROD_ACCOUNT_ID>:role/<PROD_GLUE_ROLE_NAME>",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": "glue.amazonaws.com"
        }
      }
    }
  ]
}

```

Appendix D - Proof-of-concept values and replacement map

Placeholder	Proof-of-concept value
<REGION>	us-east-1
<DEV_ACCOUNT_ID>	065320272077
<PROD_ACCOUNT_ID>	811261691581
<PROD_REPOSITORY_NAME>	glue-production
<PROD_GLUE_ASSETS_BUCKET>	aws-glue-assets-811261691581-us-east-1
<PROD_GLUE_ROLE_ARN>	arn:aws:iam::811261691581:role/Glue-role-2d766430
Dev pipeline	GlueReleasePipeline1
Dev CodeBuild	GlueReleaseBuilder
Prod CodeBuild	GlueProductionDeployer
Replacement check: Search the repository and policies for every <ANGLE_BRACKET> placeholder before committing in a real account. Also search for the proof-of-concept account IDs to confirm none remain.	

Appendix E - AWS official references

ID	Reference	URL
AWS-1	Create a CodeCommit repository	https://docs.aws.amazon.com/codecommit/latest/userguide/getting-started-cc.html
AWS-2	Using Git version control systems in AWS Glue	https://docs.aws.amazon.com/glue/latest/dg/edit-job-add-source-control-integration.html
AWS-3	Create a build project in CodeBuild	https://docs.aws.amazon.com/codebuild/latest/userguide/create-project.html
AWS-4	CodeBuild buildspec reference	https://docs.aws.amazon.com/codebuild/latest/userguide/build-spec-ref.html
AWS-5	CodeBuild action in CodePipeline and CodePipeline source behavior	https://docs.aws.amazon.com/codepipeline/latest/userguide/action-reference-CodeBuild.html
AWS-6	Cross-account resource access using IAM roles	https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies-cross-account-resource-access.html
AWS-7	CodeCommit source action in CodePipeline	https://docs.aws.amazon.com/codepipeline/latest/userguide/action-reference-CodeCommit.html
AWS-8	AWS Glue CreateJob API	https://docs.aws.amazon.com/glue/latest/webapi/API_CreateJob.html
AWS-9	AWS Glue UpdateJob API	https://docs.aws.amazon.com/glue/latest/webapi/API_UpdateJob.html
AWS-10	AWS Glue JobCommand values including glueetl	https://docs.aws.amazon.com/glue/latest/webapi/API_JobCommand.html
AWS-11	CodeCommit credential helper and temporary credentials	https://docs.aws.amazon.com/codecommit/latest/userguide/setting-up.html
AWS-12	CodeCommit returned to general availability	https://aws.amazon.com/blogs/devops/aws-codecommit-returns-to-general-availability/
AWS-13	Start CodePipeline manually	https://docs.aws.amazon.com/codepipeline/latest/userguide/pipelines-rerun-manually.html

Appendix F - final master checklist

Check	Completion criterion	Evidence / value
☐	Phase 0 replacement worksheet and dependencies complete.	
☐	Phase 1 Dev repository and Glue source control complete.	
☐	Phase 2 Prod repository, S3 and Glue role complete.	
☐	Phase 3 Dev CodeBuild role complete.	
☐	Phase 4 Prod cross-account role complete.	
☐	Phase 5 Dev release files complete.	
☐	Phase 6 Dev pipeline/build and manual trigger complete.	
☐	Phase 7 selective promotion validated.	
☐	Phase 8 Prod deployer role complete.	
☐	Phase 9 Prod framework files complete.	
☐	Phase 10 Prod CodeBuild complete.	
☐	Phase 11 create/update and runtime validation complete.	
☐	Rollback tested or tabletop reviewed.	
☐	All placeholders removed.	
☐	All change evidence retained.	